µInference: Research platform for a micro inference server for SL5

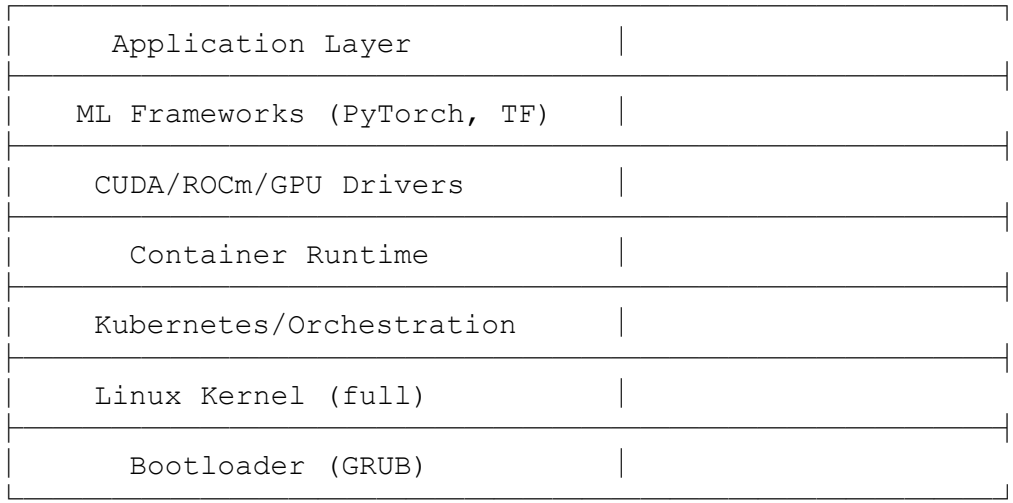
A Software Supply Chain Initiative for SL5 Security

What is the smallest stack we can build?

- From bootloader to inference in minimal lines of code
- Do we need to sacrifice functionality?

Current Inference Stack Complexity

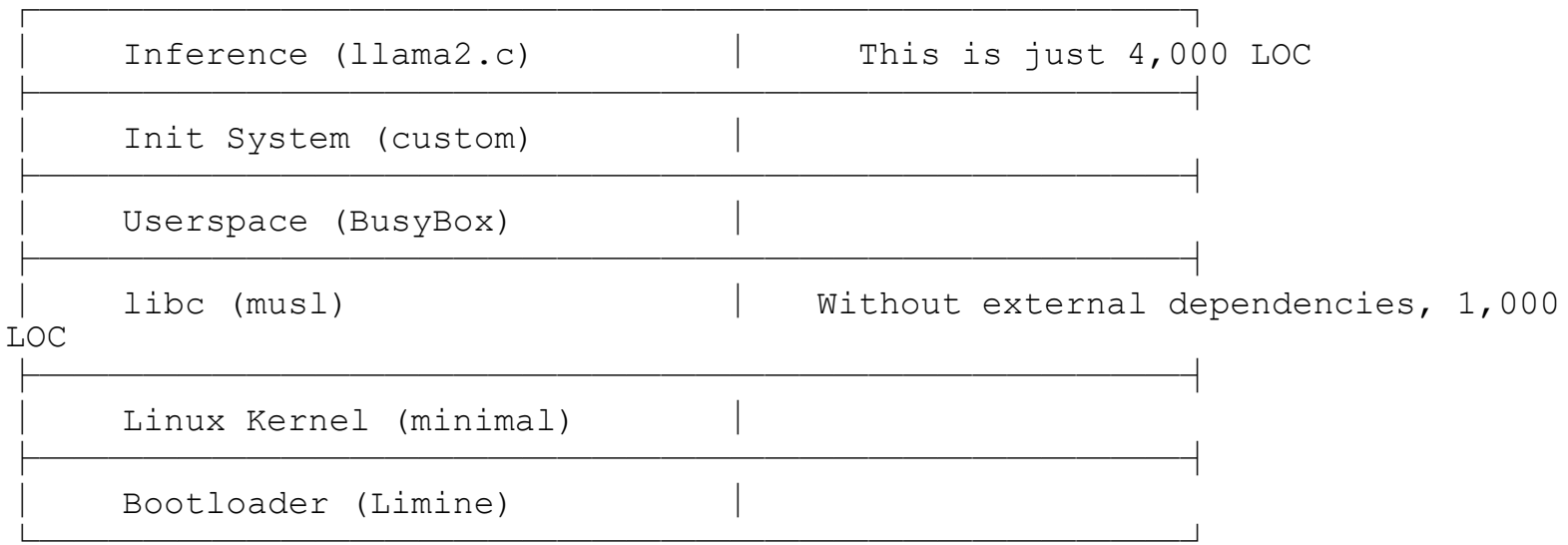
Traditional ML Infrastructure Stack



Total Estimate: Hundreds of millions of lines of code + firmware

µInference Stack

Our minimal implementation



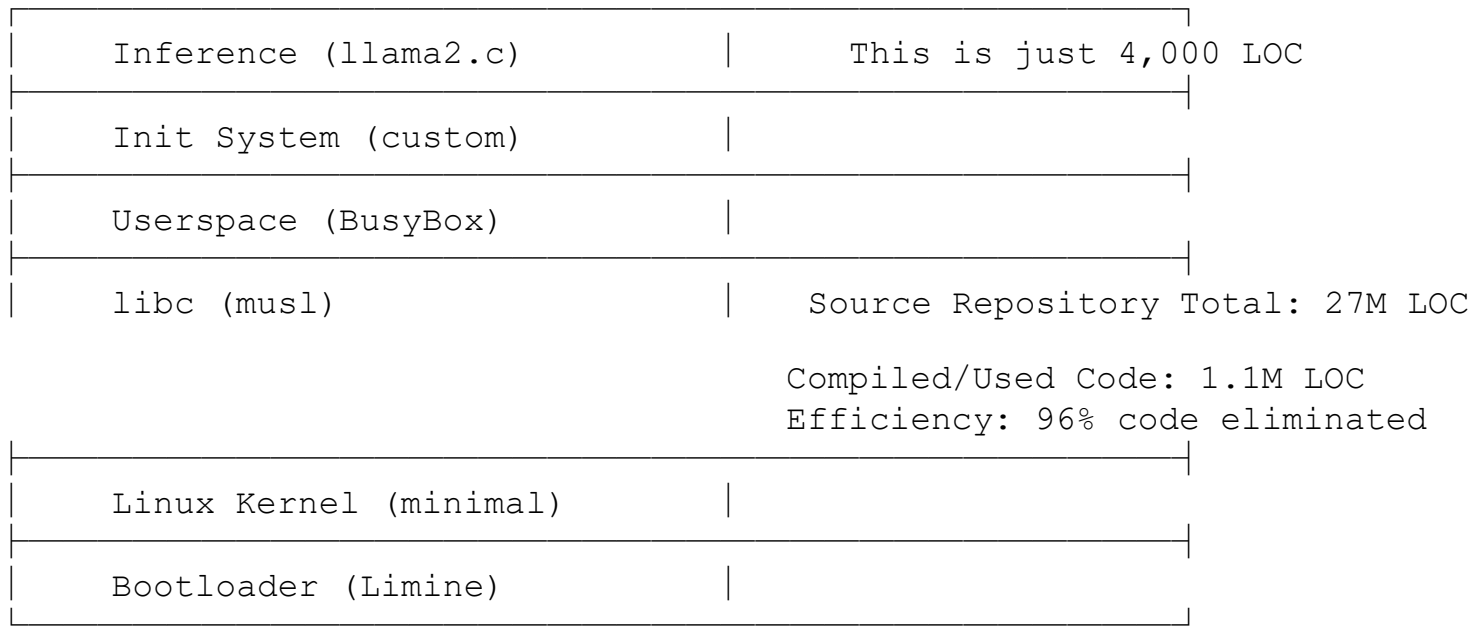
```
ubuntu@nursejoy-3:/mnt/e/Dropbox/Projects/SL5TF/muinference$ cloc .
   92955 text files.
   77979 unique files.
   17193 files ignored.

github.com/AlDanial/cloc v 1.98  T=1579.91 s (49.4 files/s, 22471.9 lines/s)
```

Language	files	blank	comment	code
C	35030	3373578	2682892	17483269
C/C++ Header	26035	736593	1425211	7265540
reStructuredText	3365	166501	68976	454780
JSON	570	2	0	385962
YAML	3608	66328	16633	310324
Assembly	1629	49410	103275	243809
Text	1890	30737	0	137667
Bourne Shell	1165	32396	22772	132521
make	2910	11824	12668	54122
SVG	74	90	1171	48177
Python	193	9257	8282	46678
Perl	72	7665	5354	38003
Rust	55	1333	8524	8077
HTML	11	2067	37	7985
yacc	11	806	432	5623
D	130	3	0	4824
PO File	6	948	1088	3733
DOS Batch	1016	26	0	3403
C++	13	550	207	3267
Bourne Again Shell	61	542	516	2571
lex	11	406	324	2540
Markdown	10	596	0	1928
awk	17	296	236	1704
Glade	2	114	0	1195
NAnt script	3	233	0	787
CSV	10	74	0	658
Linker Script	11	28	11	513
diff	7	89	264	388
m4	2	72	1	343
Cucumber	1	34	58	198
TeX	1	6	74	156
TNSDL	2	33	0	140
CSS	3	41	60	136
Windows Module Definition	2	15	0	113
Clojure	33	0	0	75
XSLT	5	13	26	61
XML	2	1	0	58
Umka	2	17	0	45
MATLAB	1	17	37	35
IDL	1	0	0	33
Jupyter Notebook	1	0	102	28
vim script	1	3	12	27
Ruby	1	4	0	25
bc	1	10	0	25
sed	2	2	11	14
INI	2	2	0	11
TOML	1	1	9	2
SUM:	77979	4492763	4359263	26651573

What Actually Ships vs Source Code

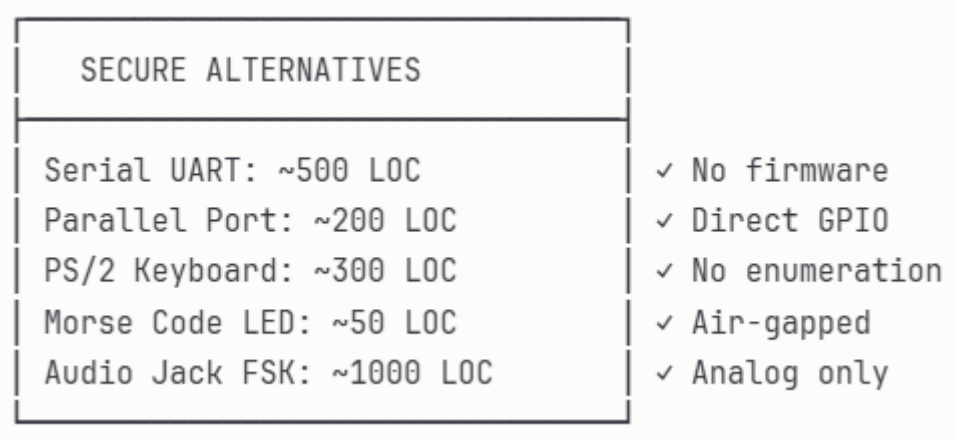
Our minimal implementation



What we removed from the kernel?

- Network stack
- Filesystems (except tmpfs)
- All drivers except essential for IO

And there is plenty of options for SL5 IO



Why Limine?

- Modern UEFI/BIOS support
- Minimal configuration
- Fast boot times
- No unnecessary features

Why musl + BusyBox?

- **musl libc**
  - 10x smaller than glibc
  - Static linking friendly
  - Clean, modern C implementation
  - No legacy baggage
- **BusyBox**
  - 300+ Unix utilities in one binary
  - Configurable feature set
  - 1MB static binary
  - Replaces coreutils, util-linux, etc.

Why llama2.c?

- Complete inference in 4,000 lines
- Pure C implementation (and easily portable to Rust with LLMs!)
- Transformer architecture
- BPE tokenizer
- Temperature sampling
- Top-p sampling
- No dependencies
- Runs models efficiently

Achieved Metrics

µInference by the Numbers

Metric	Value
ISO Size	~50MB
Boot Time	<1 seconds
RAM Usage	512MB
Inference Speed	~444 tokens/sec in consumer hardware

Reduction Ratios

- **Order of magnitude** smaller than typical ML stack
- **Order of magnitude** fewer dependencies
- **99%** less attack surface

How Much Smaller Can We Go?

- **Phase 1:**
  - Custom minimal kernel config
  - Remove more subsystems
- […]
- **Phase ?:**
  - No OS, just bootloader + inference
  - Direct hardware control

But there is also Scaling Up

llama.cpp Integration

- 4-bit quantization support
- GPU acceleration
- ~50K additional LOC
- 10-100x performance gain
- **GPU Stack**

Security Through Minimalism

Attack Surface Reduction

- 99% less code = 99% fewer bugs
- No network stack = no remote exploits (There is plenty of alternative IO) options)
- No filesystem = no persistence
- Read-only system = immutable

Where Minimal Inference Matters

- Air-gapped systems
- High-security facilities
- Research sandboxes

What This Could Prove

1. **Frontier labs don’t require millions of LOC**
   - Core inference is surprisingly simple
   - Complexity is in the ecosystem
2. **Security through simplicity is achievable**
   - Small enough to audit
3. **But there is also Scaling Up**

Research Directions

- **Immediate:**
  - Add llama.cpp backend
  - Implement secure boot
- **Medium-term:**
  - Deliberate Hardware stack
- **Long-term:**
  - ???